Transfer Learning for Pfam Classification

Ben Mindlin

June 2025

1 Overview

Understanding the relationship between amino acid sequences and protein function is a long standing problem in biology of huge practical importance. State of the art alignment based methods are unable to classify over 1/3 of known protein sequences, and deep learning models have been proposed as a potential solution to fill this gap [1]. Large scale, self supervised protein language models such as Evolutionary Scale Modelling Cambrian (ESM-C) [5] provide rich embeddings of amino acid sequences, which can be used out of the box for downstream classification tasks, including functional annotation tasks like protein family (Pfam) classification. A recent review of protein language model embeddings found that ESM models behaved better than other large scale foundation models like ProtT5, ProtBERT and SeqVec on downstream functional annotation tasks [3]. The ESM embeddings have been previously used for Pfam classification by Bugnon et al., who observed up to a 50% lower error rate than the end-to-end trained Convolutional Neural Network and Ensemble models ProtCNN and ProtENN [1] when using these embeddings as input. In this work, we take a similar transfer learning approach to Bugnon et al., but investigate the performance of classifiers trained on mean-pooled ESM embeddings instead of the full sequence of embedding vectors. This decreases the avearge size of cached embeddings by a factor of the mean sequence length (~ 155 for the Pfam dataset), which allows for smaller final stage classification models with fewer parameters.

2 Results

2.1 Initial Exploration on Reduced Size Dataset `Pfam_64`

To quickly validate methods and debug proposed models, a small test dataset of the sixty four most represented protein families was created, which we refer to as `Pfam_64`. To explore the viability of ESM embeddings, we visualised them using the standard dimensionality reduction technique of t-SNE embedding [4]. Remarkably, t-SNE embeddings from almost all classes appear to be well separated and tightly clustered, as shown in Fig 1, suggesting applying a simple classifier should be sufficient to determine each of the distinct classes. Indeed, training a multi-class logistic classifier on the 66231 sequences of `Pfam_64` achieves close to 100% accuracy, precision, recall and F1 score on the unseen test set, as shown in Table 1. This classifier was trained by using the Adam optimizer to minimize the Cross Entropy Loss over 30 epochs, with a batch size of 128 and learning rate $1e-3$.

Metric	Mean across Classes	Std across Classes
Accuracy	0.99976	Na
Precision	0.99987	0.00102
Recall	0.99971	0.00160
F1	0.99979	0.00095

Table 1: Metrics for Logistic Classifier on `Pfam_64`

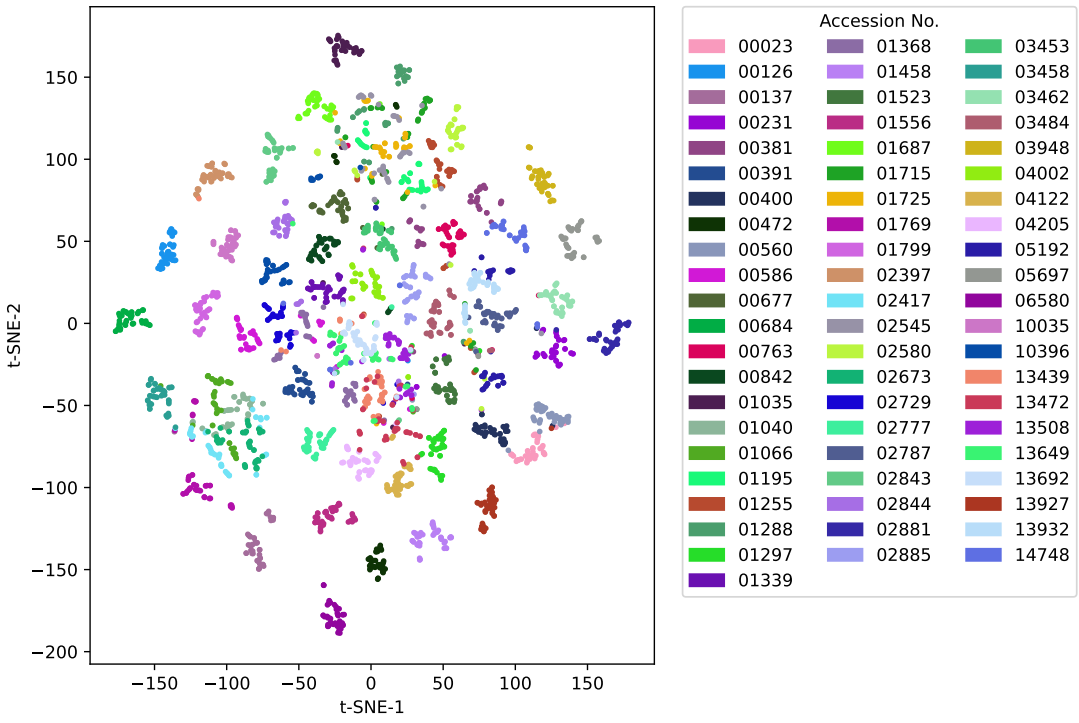

Figure 1: **t-SNE Embeddings of ESM-C Embeddings**. For fifty randomly chosen examples from each of the top 64 protein familys, ESM embeddings were calculated and mean-pooled. Two dimensional t-SNE embeddings were calculated over 5000 iterations using perplexity 5. These t-SNE embeddings coloured by true Pfam accession no. are visualized above.

2.2 Full Dataset

Following these promising initial results we scaled up to precompute embeddings for all $\sim$ 1.3 million sequences in the full training, dev and test set. These embeddings were then mean-pooled and stored, before being used to train a single layer multi-class logistic regression classifier, as described in the methods section. Using sampling to address class imbalances, the model trained to 97% accuracy but significantly overfit, leading to very low accuracies on the unseen dev and test sets. It's possible that mean-pooling is too tight of an information bottle neck to achieve high performance across this many classes. However I think more likely that a simple logistic regression architecture going from only 960 features to 17929 classes will be hard to train. Give more time and compute, I would investigate using a hierarchical classifier that first predicted clan membership and then identified family membership, using a multi layer perceptron, or using the full ESM-C embeddings.

3 Methods

Exploratory data analysis (EDA) and data cleaning can be found in the Jupyter notebook `pfamformer/scripts/pfam_eda`. Embeddings were precomputed using Google Colab T4-GPUs using the `esm` Python library [5]. Embeddings labelled by accession no. were then used to train a multi-class logistic regression classifier, using the Adam optimizer to minimize the Cross Entropy Loss over 30 epochs, with a batch size of 64 and learning rate $5e-3$. Due to the large size of the dataset, lazy loading of embeddings was investigated to attempt to reduce the strain on system RAM (see `pfamformer/data_handling.py/LazyEmbeddingDataset`. However the significant increase in io during training led to massively increased training times, and since the whole dataset was able to be read in to the 53GB of RAM allowed by Google Colab L4-GPUs, standard loading of embeddings was used. We attempted to address class imbal-

ances identified during EDA using a weighted sampler with weights inversely proportional to class frequency. Although the model trained successfully, it most likely overfit signifcantly as very poor performance was seen on the unseen test and dev sets. To address this, in future work I would decrease the weighting of under-represented classes and use stronger regularization, for example early stopping or dropout regularization.

The precomputed embeddings for the whole dataset can be found here `https://drive.google.com/drive/folders/1d3D6i8bvT0JxC0i1lbDAsVGFuPZ8rC1c?usp=drive_link`. The project code can be found here `https://github.com/bennm37/pfamformer`. The code is structured as a python package, with key scripts stored in the scripts folder.

4 Discussion

The `pfam_64` experiment highlights the power of transfer learning for functional annotation tasks in biology. The strong performance achieved using only a logistic regression classifier applied to the mean-pooled ESM-C embeddings speaks to the ability of large, self supervised protein language model's ability to learn rich, meaningful representations of protein sequences. The overfitting on the large dataset suggests that alternative architectures and stronger regularization are required. Future work could include applying more complex models such as in [2], stronger regularization and a more fine tuned approach to tackling class imbalance.

References

[1] Maxwell L. Bileschi et al. "Using deep learning to annotate the protein universe". In: *Nature Biotechnology* 40.6 (Feb. 2022), pp. 932–937. ISSN: 1546-1696. DOI: 10.1038/s41587-021-01179-w. URL: http://dx.doi.org/10.1038/s41587-021-01179-w.

[2] Leandro A. Bugnon et al. "Transfer learning: The key to functionally annotate the protein universe". In: *Patterns* 4.2 (Feb. 2023), p. 100691. ISSN: 2666-3899. DOI: 10.1016/j.patter.2023.100691. URL: http://dx.doi.org/10.1016/j.patter.2023.100691.

[3] Jia-Ying Chen et al. "Evaluating the advancements in protein language models for encoding strategies in protein function prediction: a comprehensive review". In: *Frontiers in Bioengineering and Biotechnology* 13 (Jan. 2025). ISSN: 2296-4185. DOI: 10.3389/fbioe.2025.1506508. URL: http://dx.doi.org/10.3389/fbioe.2025.1506508.

[4] Laurens van der Maaten and Geoffrey Hinton. "Visualizing Data using t-SNE". In: *Journal of Machine Learning Research* 9.86 (2008), pp. 2579–2605. URL: http://jmlr.org/papers/v9/vandermaaten08a.html.

[5] ESM Team. *ESM Cambrian: Revealing the mysteries of proteins with unsupervised learning*. https://evolutionaryscale.ai/blog/esm-cambrian. EvolutionaryScale Website. Dec. 2024. (Visited on 06/18/2025).